

This consolidated sheet includes Java MCQs, SQL queries, graph MCQs/coding, and optimized Java coding patterns.

Veeva Previous-Year Java Questions

Polished revision sheet with all options, correct answers, key explanations, and Java code for the coding questions.

Set 1: Previous-Year Java MCQs

1. Encapsulation

How is encapsulation primarily achieved in Java?

- A. By using public variables and public methods
- B. By using private variables and public getter/setter methods
- C. By using abstract classes only
- D. By utilizing multiple inheritance

Answer: B

Explanation: Encapsulation protects data by keeping variables private and providing controlled access using public getters and setters.

2. Interfaces

Which statement about interfaces in Java is correct?

- A. An interface can contain constructors
- B. Interfaces support multiple inheritance in Java
- C. Interfaces can be instantiated directly using new
- D. Interfaces can only contain final and static methods

Answer: B

Explanation: A class can implement multiple interfaces, so Java supports multiple inheritance through interfaces.

3. Method Overloading

When does overloading not occur?

- A. Same method name but different parameter number/type
- B. Same method name and same signature
- C. Same method name, same number of parameters but different type
- D. Same method name, same number and type of parameters but different signature

Answer: B

Explanation: Overloading requires a different parameter list. Same name and same signature is not overloading.

4. Inheritance

Which statement about inheritance is true?

- A. A class can extend multiple classes
- B. A subclass inherits methods from superclass
- C. Inheritance can only be achieved through interfaces
- D. Constructors are inherited by subclasses

Answer: B

Explanation: A subclass inherits accessible methods and fields from its superclass. Constructors are not inherited.

5. equals()

What does equals() usually compare?

- A. Object references
- B. Object content
- C. Class type only
- D. Memory address only

Answer: B

Explanation: equals() is intended for content comparison when properly overridden. == compares references.

6. hashCode()

Why override hashCode() when overriding equals()?

- A. To ensure object identity
- B. To maintain hash table contract
- C. To optimize memory
- D. To allow overloading

Answer: B

Explanation: Equal objects must have equal hash codes for HashMap, HashSet, and other hash-based collections.

7. Object Class

Which is true about Object class?

- A. All classes inherit from Object
- B. Object cannot be subclassed
- C. Object has no methods
- D. Object has no constructors

Answer: A

Explanation: Every Java class directly or indirectly inherits from Object.

8. protected

What is visibility of protected?

- A. Same class only
- B. Same package only
- C. Same package and subclasses
- D. Everywhere

Answer: C

Explanation: protected members are accessible within the same package and in subclasses.

9. private

What is visibility of private?

- A. Everywhere
- B. Same class only
- C. Subclasses only
- D. Same package only

Answer: B

Explanation: private members are accessible only inside the class where they are declared.

10. Multiple Inheritance

Which is true about multiple inheritance in Java?

- A. A class can implement multiple interfaces
- B. A class can extend multiple classes
- C. Multiple inheritance is allowed for classes and interfaces

D. A class can inherit from multiple abstract classes

Answer: A

Explanation: Java does not allow a class to extend multiple classes, but it can implement multiple interfaces.

11. Integer Cache

What is the output?

```
Integer num1 = 100;
Integer num2 = 100;
Integer num3 = 500;
Integer num4 = 500;

System.out.println(num1 == num2);
System.out.println(num3 == num4);
```

- A. true true
- B. true false
- C. false true
- D. false false

Answer: B

Explanation: Integer values from -128 to 127 are cached. 100 uses cache, but 500 creates separate objects.

12. Byte Array To String

What is the output?

```
byte[] arr = {97, 98, 99, 100, 101};
String str = new String(arr);
System.out.println(str);
```

- A. abcde
- B. 979899100101
- C. Compile-time error
- D. Runtime error

Answer: A

Explanation: ASCII values 97, 98, 99, 100, 101 correspond to a, b, c, d, e.

13. charAt()

What is the output?

```
String str = "Java Programming";
char ch = str.charAt(2);
System.out.println(ch);
```

- A. J
- B. a
- C. v
- D. P

Answer: C

Explanation: Indexing starts at 0. J=0, a=1, v=2.

Set 1: Coding Questions With Java Code

Coding 1. Closest Number To Target

Given an array and target K, return the number with the smallest absolute difference from K. If there is a tie, choose the greater number.

```
class Main {
    public static void main(String[] args) {
        int[] arr = {9, 11, 5, 3, 25, 18};
        int k = 6;

        int closest = arr[0];
        int minDiff = Math.abs(arr[0] - k);

        for (int i = 1; i < arr.length; i++) {
            int diff = Math.abs(arr[i] - k);

            if (diff < minDiff || (diff == minDiff && arr[i] > closest)) {
```

```

        minDiff = diff;
        closest = arr[i];
    }
}

System.out.println(closest);
}
}

```

Time: O(n)

Space: O(1)

Coding 2. Validate Subsequence

Check whether sequence is a subsequence of array. Elements must appear in the same order, but not necessarily continuously.

```

class Main {
    public static void main(String[] args) {
        int[] array = {5, 1, 22, 25, 6, -1, 8, 10};
        int[] sequence = {1, 6, -1, 10};

        int i = 0;

        for (int x : array) {
            if (i < sequence.length && x == sequence[i]) {
                i++;
            }
        }

        System.out.println(i == sequence.length);
    }
}

```

Time: O(n)

Space: O(1)

Coding 3. First Non-Repeating Character

Find the first character that appears only once. If none exists, print -1.

Frequency array version for lowercase English letters:

```

class Main {
    public static void main(String[] args) {
        String s = "loveleetcde";

        int[] freq = new int[26];

        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }

        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                System.out.println(s.charAt(i));
                System.out.println(i);
                return;
            }
        }

        System.out.println(-1);
    }
}

```

HashMap version for general characters:

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        String s = "loveleetcde";

        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : s.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (map.get(ch) == 1) {
                System.out.println(ch);
                System.out.println(i);
                return;
            }
        }

        System.out.println(-1);
    }
}

```

Time: O(n)

Space: O(1) for frequency array, O(k) for HashMap

Set 2: OOP Java MCQs

1. Interface Implementation

When a class implements an interface, which of the following must be implemented?

- A. Only abstract methods
- B. Only non-abstract methods
- C. Both abstract and non-abstract methods
- D. Only static methods

Answer: A

Explanation: A class implementing an interface must implement its abstract methods.

2. Method Overriding

What happens if a method in a subclass has the same name and parameters as a method in its superclass?

- A. The superclass method is executed
- B. The subclass method is executed
- C. A compile-time error occurs
- D. The program fails to run

Answer: B

Explanation: The subclass method overrides the superclass method.

3. Java Interface

Which of the following correctly describes a Java interface?

- A. It can have method implementations
- B. It can contain instance variables
- C. It can extend multiple classes
- D. It can only contain abstract methods

Answer: D

Explanation: This is the old/basic MCQ answer. Modern Java interfaces can also have default and static methods.

4. Serializable

What is the purpose of the Serializable interface in Java?

- A. To create a deep copy of an object
- B. To allow objects to be serialized
- C. To enable thread safety
- D. To implement cloning

Answer: B

Explanation: Serializable marks a class so its objects can be converted into a byte stream.

5. final Method

How do you prevent a method from being overridden in a subclass?

- A. Declare it as final
- B. Declare it as private
- C. Declare it as static
- D. Declare it as abstract

Answer: A

Explanation: A final method cannot be overridden.

6. Multiple Interfaces

Which statement about multiple inheritance of interfaces is true?

- A. It is allowed in Java
- B. It is not allowed in Java
- C. It leads to ambiguity always
- D. Only one interface can be inherited

Answer: A

Explanation: A class can implement multiple interfaces.

7. Inner Class

Which keyword is used to declare an inner class?

- A. inner
- B. static
- C. public
- D. class

Answer: D

Explanation: Inner classes are declared with the normal class keyword.

8. Invalid Casting

What is the result of attempting to cast an object to a class that it does not inherit from?

- A. Compile-time error
- B. Run-time exception
- C. It will return null
- D. It will create a new instance

Answer: B

Explanation: Invalid downcasting causes ClassCastException at runtime.

9. serialVersionUID

In object serialization, what is serialVersionUID?

- A. A unique identifier for each class version
- B. A method used for serialization
- C. A variable that holds serialized data
- D. A marker interface

Answer: A

Explanation: serialVersionUID is used during deserialization to verify class compatibility.

10. Marker Interface

What is the purpose of a marker interface in Java?

- A. To provide methods for implementation
- B. To mark a class for specific behavior
- C. To prevent multiple inheritance
- D. To enforce type safety

Answer: B

Explanation: Marker interfaces have no methods. They mark classes for special behavior.

11. static Members

Which statement about static members is true?

- A. Static members are associated with the instance of a class

- B. Static members can be accessed without creating an instance of a class
- C. Static members cannot be inherited
- D. Static members can only be private

Answer: B

Explanation: Static members belong to the class, not an individual object.

12. Blank Final Variable

What is the output?

```
class Test {
    public final int a;
}

class Example {
    public static void main(String args[]) {
        Test obj = new Test();
        System.out.println(obj.a);
    }
}
```

- A. 0
- B. Garbage value
- C. Compile time error: variable is not initialized
- D. Run time error: a is the blank variable

Answer: C

Explanation: a is a blank final variable and must be initialized.

13. Method Or Constructor

What is the output?

```
class Example {
    private int x;

    public static void main(String args[]) {
        Example obj = new Example();
    }

    public void Example(int x) {
        System.out.println(x);
    }
}
```

- A. 0
- B. Garbage value
- C. Compile time error
- D. No output / blank screen

Answer: D

Explanation: public void Example(int x) is a normal method, not a constructor.

14. Constructor Selection

What is the output?

```
class Example {
    private int x;

    public static void main(String args[]) {
        Example obj = new Example(5);
    }

    public Example(int x) {
        System.out.println("x = " + x);
    }

    public void Example(int x) {
        System.out.println(x);
    }
}
```

- A. x = 5
- B. 5
- C. Compile time error: ambiguous call of Example(int)
- D. Run time error

Answer: A

Explanation: public Example(int x) is the constructor. The void version is a normal method.

15. String Pool And equals()

What is the output?

```
class Test {
    public static void main(String args[]) {
        String str1 = new String("Hello World");
        String str2 = new String("Hello World");

        String str3 = "Hello World";
        String str4 = "Hello World";

        int a = 0, b = 0, c = 0;

        if (str3 == str4)
            a = 1;
        else
            a = 2;

        if (str1.equals(str3))
            b = 1;
        else
            b = 2;

        if (str1 == str4)
            c = 1;
        else
            c = 2;

        System.out.println("a= " + a + " b= " + b + " c= " + c);
    }
}
```

- A. a=2 b=1 c=2
- B. a=2 b=2 c=2
- C. a=1 b=2 c=1
- D. a=1 b=1 c=2

Answer: D

Explanation: str3 and str4 point to the same string pool object. equals compares content. str1 and str4 are different references.

16. String Object Count

How many String objects are created in the above example?

- A. 1
- B. 2
- C. 3
- D. 4

Answer: C

Explanation: One string pool literal plus two new String objects.

17A. HashMap

What is the output?

```
import java.util.HashMap;
import java.util.Map;

public class MyClass {
    public static void main(String args[]) {
        Map<String, String> hashMap = new HashMap<String, String>();
        hashMap.put(new String("a"), "audi");
        hashMap.put(new String("a"), "ferrari");
        System.out.println(hashMap);
    }
}
```

- A. {a=audi}
- B. {a=ferrari}
- C. {a=audi, a=ferrari}
- D. Compile-time error

Answer: B

Explanation: HashMap compares keys using equals() and hashCode(), so the second value replaces the first.

17B. IdentityHashMap

What is the output?

```
import java.util.IdentityHashMap;
import java.util.Map;

public class MyClass {
    public static void main(String args[]) {
        Map<String, String> identityHashMap = new IdentityHashMap<String, String>();
        identityHashMap.put(new String("a"), "audi");
        identityHashMap.put(new String("a"), "ferrari");
        System.out.println(identityHashMap);
    }
}
```

- A. {a=ferrari}
- B. {a=audi}
- C. Both entries remain, such as {a=audi, a=ferrari}; order may vary
- D. Runtime exception

Answer: C

Explanation: IdentityHashMap compares keys using ==, not equals(), so both new String objects are treated as different keys.

Veeva Java MCQ and SQL Revision

This sheet contains important Java output-based MCQs and 7 SQL query patterns for Veeva preparation.

Java MCQs

1. String Immutability

What is the output?

```
public class Test {
    public static void main(String[] args) {
        String s = "abc";
        s.toUpperCase();
        System.out.println(s);
    }
}
```

- A. ABC
- B. abc
- C. Compile-time error
- D. Runtime error

Answer: B

Explanation: String is immutable. toUpperCase() returns a new string, but it is not assigned back to s.

2. Method Overloading With null

What is the output?

```
public class Code {
    public static void main(String[] args) {
        method(null);
    }

    public static void method(Object o) {
        System.out.println("Object method");
    }

    public static void method(String s) {
        System.out.println("String method");
    }
}
```

- A. Object method
- B. String method
- C. Compile-time error
- D. Runtime error

Answer: B

Explanation: String is more specific than Object, so method(String s) is selected.

3. String Concatenation

What is the output?

```
public class Test {
    public static void main(String[] args) {
        String s = new String("5");
        System.out.println(1 + 10 + s + 1 + 10);
    }
}
```

- A. 115110
- B. 111510
- C. 27
- D. 151110

Answer: A

Explanation: 1 + 10 is evaluated first as 11. After string appears, remaining + operations become concatenation.

4. Static Block, Instance Block, Constructor

What is the output?

```
class A {
    static {
        System.out.print("1");
    }

    {
        System.out.print("2");
    }

    public A() {
        System.out.print("3");
    }
}

public class Test {
    public static void main(String[] args) {
        A a1 = new A();
        A a2 = new A();
    }
}
```

- A. 12323
- B. 121323
- C. 2323
- D. 112233

Answer: A

Explanation: Static block runs once. For each object, instance block runs before constructor.

5. Instance Method Overloading With null

What is the output?

```
public class Test {
    public static void main(String[] args) {
        new Test().print(null);
    }

    public void print(Object o) {
        System.out.println("Object");
    }

    public void print(String s) {
        System.out.println("String");
    }
}
```

- A. Object
- B. String
- C. Compile-time error
- D. Runtime error

Answer: B

Explanation: String is more specific than Object.

6. Floating Point Precision

What is the output?

```
public class Test {
    public static void main(String[] args) {
        double a = 0.1 + 0.2;
    }
}
```

```

        System.out.println(a == 0.3);
        System.out.println(a);
    }
}

```

- A. true and 0.3
- B. false and 0.30000000000000004
- C. true and 0.30000000000000004
- D. Compile-time error

Answer: B

Explanation: Decimal floating-point values are not always represented exactly in binary.

7. try-catch-finally

What is the output?

```

public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("A");
            throw new RuntimeException("Test");
        } catch (RuntimeException e) {
            System.out.println("B");
        } finally {
            System.out.println("C");
        }
    }
}

```

- A. A
- B. A B
- C. A B C
- D. A C

Answer: C

Explanation: The exception is caught, and finally always executes.

8. Varargs Overloading

What is the output?

```

public class Test {
    public static void main(String[] args) {
        new Test().print(1, 2, 3);
    }

    public void print(int... numbers) {
        System.out.println("int...");
    }

    public void print(Integer i1, Integer i2) {
        System.out.println("Integer");
    }
}

```

- A. int...
- B. Integer
- C. Compile-time error
- D. Runtime error

Answer: A

Explanation: The call has 3 arguments. `print(Integer, Integer)` accepts only 2 arguments, so `varargs` is selected.

9. PriorityQueue

What is the output?

```

import java.util.*;

public class priorityQueue {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue<>();
        queue.add(11);
        queue.add(10);
        queue.add(22);
        queue.add(5);
        queue.add(12);
        queue.add(2);

        while (queue.isEmpty() == false)
            System.out.printf("%d ", queue.remove());

        System.out.println();
    }
}

```

```
}  
}
```

- A. 11 10 22 5 12 2
- B. 2 12 5 22 10 11
- C. 2 5 10 11 12 22
- D. 22 12 11 10 5 2

Answer: C

Explanation: Default `PriorityQueue<Integer>` is a min-heap, so removal gives ascending order.

10. TreeSet

What is the output?

```
import java.util.*;  
  
public class TreeSet {  
    public static void main(String[] args) {  
        TreeSet<String> treeSet = new TreeSet<>();  
  
        treeSet.add("Geeks");  
        treeSet.add("For");  
        treeSet.add("Geeks");  
        treeSet.add("GeeksforGeeks");  
  
        for (String temp : treeSet)  
            System.out.printf(temp + " ");  
  
        System.out.println();  
    }  
}
```

- A. Geeks For Geeks GeeksforGeeks
- B. Geeks For GeeksforGeeks
- C. For Geeks GeeksforGeeks
- D. For GeeksforGeeks Geeks

Answer: C

Explanation: `TreeSet` removes duplicates and stores elements in sorted order.

11. LinkedList removeAll

What is the output?

```
import java.util.*;  
  
public class LinkedList {  
    public static void main(String[] args) {  
        List<String> list1 = new LinkedList<>();  
        list1.add("Geeks");  
        list1.add("For");  
        list1.add("Geeks");  
        list1.add("GFG");  
        list1.add("GeeksforGeeks");  
  
        List<String> list2 = new LinkedList<>();  
        list2.add("Geeks");  
  
        list1.removeAll(list2);  
  
        for (String temp : list1)  
            System.out.printf(temp + " ");  
  
        System.out.println();  
    }  
}
```

- A. For Geeks GFG GeeksforGeeks
- B. For GeeksforGeeks GFG
- C. For GFG for
- D. For GFG GeeksforGeeks

Answer: D

Explanation: `removeAll` removes all occurrences of "Geeks" from `list1`.

12. Iterator Type Mismatch

What is the output?

```
import java.util.*;  
  
public class stack {
```

```

    public static void main(String[] args) {
        List<String> list = new LinkedList<>();
        list.add("Geeks");
        list.add("For");
        list.add("Geeks");
        list.add("GeeksforGeeks");
        Iterator<Integer> iter = list.iterator();

        while (iter.hasNext())
            System.out.printf(iter.next() + " ");

        System.out.println();
    }
}

```

- A. Geeks For Geeks GeeksforGeeks
- B. GeeksforGeeks Geeks For Geeks
- C. Runtime Error
- D. Compilation Error

Answer: D

Explanation: `list.iterator()` returns `Iterator<String>`, not `Iterator<Integer>`.

13. Private Constructor And Private Field

What is the output?

```

class Helper {
    private int data;

    private Helper() {
        data = 5;
    }
}

public class Test {
    public static void main(String[] args) {
        Helper help = new Helper();
        System.out.println(help.data);
    }
}

```

- A. Compilation error
- B. 5
- C. Runtime error
- D. None of these

Answer: A

Explanation: The constructor is private and data is private, so both accesses fail outside `Helper`.

14. Runnable Constructor Syntax

What is the output?

```

public class Test implements Runnable {
    public void run() {
        System.out.printf(" Thread's running ");
    }

    try {
        public Test() {
            Thread.sleep(5000);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    public static void main(String[] args) {
        Test obj = new Test();
        Thread thread = new Thread(obj);
        thread.start();
        System.out.printf(" GFG ");
    }
}

```

- A. GFG Thread's running
- B. Thread's running GFG
- C. Compilation error
- D. Runtime error

Answer: C

Explanation: A constructor cannot be declared inside a `try` block.

7 SQL Query Patterns

1. 2nd Highest Salary In Engineering Department

If more than one person shares the highest salary, select the next distinct salary.

```
SELECT MAX(e.salary) AS salary
FROM employees e
JOIN departments d
ON e.department_id = d.id
WHERE d.name = 'engineering'
AND e.salary < (
    SELECT MAX(e2.salary)
    FROM employees e2
    JOIN departments d2
    ON e2.department_id = d2.id
    WHERE d2.name = 'engineering'
);
```

Pattern: MAX(salary) less than the highest salary.

2. Three-Day Rolling Average For Deposits By Day

```
WITH daily_deposits AS (
    SELECT
        DATE_FORMAT(created_at, '%Y-%m-%d') AS dt,
        SUM(transaction_value) AS daily_total
    FROM bank_transactions
    WHERE transaction_value > 0
    GROUP BY DATE_FORMAT(created_at, '%Y-%m-%d')
)
SELECT
    dt,
    AVG(daily_total) OVER (
        ORDER BY dt
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_three_day
FROM daily_deposits
ORDER BY dt;
```

Pattern: daily aggregation first, then window function.

3. Customers With More Than 3 Transactions In Both 2019 And 2020

```
SELECT u.name AS customer_name
FROM users u
JOIN transactions t
ON u.id = t.user_id
WHERE YEAR(t.created_at) IN (2019, 2020)
GROUP BY u.id, u.name
HAVING SUM(CASE WHEN YEAR(t.created_at) = 2019 THEN 1 ELSE 0 END) > 3
AND SUM(CASE WHEN YEAR(t.created_at) = 2020 THEN 1 ELSE 0 END) > 3;
```

Pattern: conditional count using SUM(CASE WHEN ...).

4. Histogram Of Comments Per User In January 2020

Users with no January comments must be counted in the 0 bucket.

```
WITH user_comment_counts AS (
    SELECT
        u.id,
        COUNT(c.user_id) AS comment_count
    FROM users u
    LEFT JOIN comments c
    ON u.id = c.user_id
    AND c.created_at >= '2020-01-01'
    AND c.created_at < '2020-02-01'
    GROUP BY u.id
)
SELECT
    comment_count,
    COUNT(*) AS frequency
FROM user_comment_counts
GROUP BY comment_count
ORDER BY comment_count;
```

Pattern: count per user first, then group by the count.

5. Top 3 Departments By Percentage Of Employees Over 100K

Only include departments with at least 10 employees.

```
SELECT
    d.name AS department_name,
    COUNT(e.id) AS number_of_employees,
    AVG(CASE WHEN e.salary > 100000 THEN 1.0 ELSE 0.0 END) AS percentage_over_100k
FROM departments d
JOIN employees e
ON d.id = e.department_id
GROUP BY d.id, d.name
HAVING COUNT(e.id) >= 10
ORDER BY percentage_over_100k DESC
LIMIT 3;
```

Pattern: percentage = average of 1s and 0s.

6. Employees Above Average Salary

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Pattern: subquery in WHERE.

7. Third Transaction Per User

```
WITH ranked_transactions AS (
    SELECT
        user_id,
        spend,
        transaction_date,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY transaction_date ASC
        ) AS rn
    FROM transactions
)
SELECT user_id, spend, transaction_date
FROM ranked_transactions
WHERE rn = 3;
```

Pattern: ROW_NUMBER() with PARTITION BY.

SQL Memory Hooks

- Second highest distinct salary: MAX(salary) WHERE salary < MAX(salary)
- Rolling average: AVG(...) OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
- Conditional count: SUM(CASE WHEN condition THEN 1 ELSE 0 END)
- Histogram: count per entity first, then group by that count
- Percentage: AVG(CASE WHEN condition THEN 1.0 ELSE 0.0 END)
- Ranking: ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)

New Java MCQs - July 12

15. String Concatenation Order

What is the output?

```
public class StringTest {
    public static void main(String[] args) {
        System.out.print(10 + 20 + "Java" + 10 + 20);
    }
}
```

- A. 1020Java1020
- B. 30Java30
- C. 30Java1020
- D. 1020Java30

Answer: C

Explanation: 10 + 20 is numeric addition first, giving 30. After "Java", the remaining + operations become string concatenation.

16. Integer Cache

What is the output?

```
public class CacheTest {
    public static void main(String[] args) {
        Integer num1 = 100;
        Integer num2 = 100;
        Integer num3 = 150;
        Integer num4 = 150;

        System.out.print((num1 == num2) + " " + (num3 == num4));
    }
}
```

- A. true true

- B. false false
- C. true false
- D. false true

Answer: C

Explanation: Integer values from -128 to 127 are cached. 100 uses cache, but 150 does not.

17. finally Return Value

What is the output?

```
public class ExceptionTest {
    public static int getValue() {
        try {
            return 1;
        } catch (Exception e) {
            return 2;
        } finally {
            return 3;
        }
    }

    public static void main(String[] args) {
        System.out.print(getValue());
    }
}
```

- A. 1
- B. 2
- C. 3
- D. Compilation Error

Answer: C

Explanation: A return inside finally overrides the return from try or catch.

18. Overloading With null

What is the output?

```
public class OverloadTest {
    public static void testMethod(Object o) {
        System.out.print("Object");
    }

    public static void testMethod(String s) {
        System.out.print("String");
    }

    public static void main(String[] args) {
        testMethod(null);
    }
}
```

- A. Object
- B. String
- C. Compilation Error
- D. NullPointerException

Answer: B

Explanation: String is more specific than Object, so the String overload is selected.

19. Character Addition

What is the output?

```
public class Test {
    public static void main(String[] args) {
        System.out.println('j' + 'a' + 'v' + 'a');
    }
}
```

- A. java
- B. 418
- C. Compilation Error
- D. 414

Answer: B

Explanation: Characters are added using their Unicode values: j=106, a=97, v=118, a=97. Total = 418.

20. Integer Object Comparison

What is the output?

```
public class Test {
    public static void main(String[] args) {
        Integer num1 = 400;
        Integer num2 = 400;
        if (num1 == num2) {
            System.out.println(0);
        } else {
            System.out.println(1);
        }
    }
}
```

- A. 0
- B. 1
- C. Compilation Error
- D. Runtime Error

Answer: B

Explanation: 400 is outside Integer cache range, so num1 and num2 are different objects. == compares references.

21. Unary Operators

What is the output?

```
public class Test {
    public static void main(String[] args) {
        int i = 20 + +9 - -12 + +4 - -13 + +19;
        System.out.println(i);
    }
}
```

- A. 77
- B. 51
- C. 29
- D. Compilation Error

Answer: A

Explanation: + +9 means +9 and - -12 means +12. So 20 + 9 + 12 + 4 + 13 + 19 = 77.

22. String Concatenation Lines

What is the output?

```
public class Test {
    public static void main(String[] args) {
        System.out.println(10 + 20 + "Java");
        System.out.println("Java" + 10 + 20);
    }
}
```

- A. 30Java and Java1020
- B. 1020Java and Java30
- C. 30Java and Java30
- D. 1020Java and Java1020

Answer: A

Explanation: Before a string, + does numeric addition. After a string, + does concatenation.

23. System.exit And finally

What is the output?

```
public class FinallyBlockTest {
    public static void main(String[] args) {
        try {
            System.out.println("Inside try block");
            System.exit(0);
        } catch (Exception e) {
            System.out.println("Inside catch block");
        } finally {
            System.out.println("Inside finally block");
        }
    }
}
```

- A. Inside try block, then Inside finally block

- B. Inside try block only
- C. Inside catch block, then Inside finally block
- D. Compilation Error

Answer: B

Explanation: `System.exit(0)` terminates the JVM, so finally does not execute.

24. Division By Zero With double And int

What is the output?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(1.0 / 0);  
        System.out.println(0.0 / 0);  
        System.out.println(0 / 0);  
    }  
}
```

- A. Infinity, NaN, then ArithmeticException
- B. ArithmeticException immediately
- C. Infinity, 0.0, 0
- D. Compilation Error

Answer: A

Explanation: Floating-point division by zero gives Infinity or NaN. Integer division by zero throws ArithmeticException.

25. Multiplication Before String Concatenation

What is the output?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(10 * 20 + "Hello");  
        System.out.println("Hello" + 10 * 20);  
    }  
}
```

- A. 200Hello and Hello200
- B. 1020Hello and Hello1020
- C. 200Hello and Hello1020
- D. Compilation Error

Answer: A

Explanation: Multiplication has higher precedence than string concatenation, so `10 * 20` becomes 200 in both lines.

Veeva Graph Questions and Coding Practice

Polished revision sheet for graph MCQs, BFS/DFS patterns, graph coding tasks, and two extra HashMap/String parsing problems.

Part 1: Graph MCQs With Options

1. Counting Graph Edges

In the given graph with nodes 0, 1, 2, 3, 4, and 5, count the total number of edges.

- A. Total Edges = 6
- B. Total Edges = 7
- C. Total Edges = 8
- D. Total Edges = 9

Answer: C

Explanation: The edges are 2-3, 2-4, 2-0, 0-4, 0-5, 4-5, 4-1, and 5-1. Total edges = 8.

2. Maximum Edges In Connected Graph

In a graph with n vertices and m edges, what is the maximum number of edges that can be present in a connected simple undirected graph?

- A. n
- B. $n - 1$
- C. $n(n - 1) / 2$
- D. $2 * n$

Answer: C

Explanation: A simple undirected graph has maximum edges when it is complete. Formula = $n(n - 1) / 2$.

3. Connected Graph With No Cycles

A connected graph with no cycles is known as:

- A. Complete Graph
- B. Bipartite Graph
- C. Tree
- D. Cyclic Graph

Answer: C

Explanation: A tree is a connected acyclic graph.

4. Disconnected Graph

In a disconnected graph, what can be present between two nodes from different components?

- A. A unique path
- B. A cycle
- C. No connecting path
- D. Weighted edges

Answer: C

Explanation: In a disconnected graph, at least two nodes exist such that no path connects them.

5. Graph Nodes And Edges Count

In the shown graph with four red nodes and all possible connections between them, what are the total nodes and total edges?

- A. Total Nodes = 3, Total Edges = 5
- B. Total Nodes = 4, Total Edges = 6
- C. Total Nodes = 4, Total Edges = 4
- D. Total Nodes = 5, Total Edges = 6

Answer: B

Explanation: There are 4 nodes. It is a complete graph K_4 , so edges = $4 * 3 / 2 = 6$.

6. Adjacency Matrix Definition

In an adjacency matrix of an unweighted graph, what does $matrix[i][j]$ represent?

- A. The label of node i
- B. The presence or absence of an edge between nodes i and j
- C. The weight of the edge between nodes i and j
- D. The number of nodes in the graph

Answer: B

Explanation: For an unweighted graph, $matrix[i][j]$ is usually 1 if edge $i-j$ exists, otherwise 0.

7. Adjacency Matrix Characteristic

For an undirected graph, what is a characteristic of the adjacency matrix?

- A. It is asymmetric
- B. It is a sparse matrix
- C. It is a symmetric matrix

D. It contains only 0s

Answer: C

Explanation: If u is connected to v , then v is also connected to u . So $\text{matrix}[u][v] = \text{matrix}[v][u]$.

8. Sparse Graph Representation

Which graph representation is more memory-efficient for sparse graphs?

- A. Adjacency Matrix
- B. Adjacency List
- C. Both have the same memory efficiency
- D. Edge List

Answer: B

Explanation: Adjacency matrix uses $O(V^2)$ memory. Adjacency list uses $O(V + E)$, which is better for sparse graphs.

9. BFS Node Traversal Order

In Breadth-First Search (BFS), how are nodes visited in a graph?

- A. In a depth-first manner
- B. In a random order
- C. In a level-by-level manner
- D. In a reverse order

Answer: C

Explanation: BFS visits all nodes at the current level before moving to the next level.

10. Connected Components Meaning

In a graph with four connected components, what does it mean?

- A. There are four isolated vertices
- B. The graph is disconnected into four separate groups of vertices
- C. There are four cycles in the graph
- D. Each vertex is connected to exactly four other vertices

Answer: B

Explanation: A connected component is a group of vertices where every vertex is reachable from every other vertex in the same group.

11. Single Vertex Component

Can a single vertex be considered a connected component in a graph?

- A. Yes
- B. No
- C. Only in directed graphs
- D. Only in weighted graphs

Answer: A

Explanation: An isolated vertex is also a connected component by itself.

Part 2: Rearrangement Questions

12. Rearrange DFS Steps

Rearrange the following DFS steps:

- A. Start from a source node
- B. If a node has no unvisited neighbors, backtrack to the previous node
- C. Recursively explore each unvisited neighbor
- D. Visit the current node and mark it visited using a boolean array/vector

Answer: A, D, C, B

Explanation: DFS starts at a source, visits and marks it, explores unvisited neighbors recursively, and backtracks when needed.

13. Rearrange BFS Steps

Rearrange the following BFS steps:

- A. Enqueue all unvisited neighbors of the dequeued node into the queue and mark them as visited
- B. While the queue is not empty
- C. Start from a source node
- D. Enqueue the source node into a queue and mark it as visited
- E. Dequeue a node from the queue and process it
- F. Repeat until the queue is empty

Answer: C, D, B, E, A, F

Explanation: BFS uses a queue. It starts from a source, enqueues it, then repeatedly dequeues and visits unvisited neighbors.

Part 3: Graph Implementation And Coding Patterns

14. Build Adjacency Matrix

Task: Read n nodes and m undirected edges, then print the adjacency matrix.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[][] adjMatrix = new int[n + 1][n + 1];

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            adjMatrix[u][v] = 1;
            adjMatrix[v][u] = 1;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                System.out.print(adjMatrix[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Time: $O(n^2 + m)$

Space: $O(n^2)$

15. Build Adjacency List

Task: Read n nodes and m undirected edges, then print the adjacency list.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        ArrayList<Integer>[] graph = new ArrayList[n + 1];

        for (int i = 1; i <= n; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph[u].add(v);
            graph[v].add(u);
        }

        for (int i = 1; i <= n; i++) {
            for (int node : graph[i]) {
                System.out.print(node + " ");
            }
        }
    }
}
```

```

        System.out.println();
    }
    sc.close();
}
}

```

Time: $O(n + m)$

Space: $O(n + m)$

16. DFS Traversal

Task: Traverse graph using Depth-First Search.

```

import java.util.*;

class Main {
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int nei : graph.get(node)) {
            if (!visited[nei]) {
                dfs(nei, visited, graph);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        boolean[] visited = new boolean[n + 1];

        for (int i = 1; i <= n; i++) {
            if (!visited[i]) {
                dfs(i, visited, graph);
            }
        }

        sc.close();
    }
}

```

Time: $O(n + m)$

Space: $O(n + m)$

17. BFS Traversal

Task: Traverse graph using Breadth-First Search from node 1.

```

import java.util.*;

class Main {
    static void bfs(int start, boolean[] visited, List<List<Integer>> graph) {
        Queue<Integer> q = new LinkedList<>();

        q.add(start);
        visited[start] = true;

        while (!q.isEmpty()) {
            int node = q.poll();
            System.out.print(node + " ");

            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.add(nei);
                }
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

```

```

        graph.get(u).add(v);
        graph.get(v).add(u);
    }

    boolean[] visited = new boolean[n + 1];
    bfs(1, visited, graph);

    sc.close();
}
}

```

Time: $O(n + m)$

Space: $O(n + m)$

18. Check Reachability From Node 1 In Directed Graph

Task: Print all vertices reachable from node 1 in increasing order.

```

import java.util.*;

class Main {
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph) {
        visited[node] = true;

        for (int nei : graph.get(node)) {
            if (!visited[nei]) {
                dfs(nei, visited, graph);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int a = sc.nextInt();
            int b = sc.nextInt();
            graph.get(a).add(b);
        }

        boolean[] visited = new boolean[n + 1];
        dfs(1, visited, graph);

        for (int i = 1; i <= n; i++) {
            if (visited[i]) {
                System.out.print(i + " ");
            }
        }

        sc.close();
    }
}

```

Time: $O(n + m)$

Space: $O(n + m)$

19. Chef Shortest Route

Task: Find minimum number of servers in a route from node 1 to node n in an undirected unweighted graph. Return -1 if unreachable.

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int a = sc.nextInt();
            int b = sc.nextInt();
            graph.get(a).add(b);
            graph.get(b).add(a);
        }

        int[] dist = new int[n + 1];
        Arrays.fill(dist, -1);

        Queue<Integer> q = new LinkedList<>();
        q.add(1);
        dist[1] = 0;

        while (!q.isEmpty()) {
            int node = q.poll();

            for (int nei : graph.get(node)) {
                if (dist[nei] == -1) {

```

```

        dist[nei] = dist[node] + 1;
        q.add(nei);
    }
}
}

if (dist[n] == -1) {
    System.out.println(-1);
} else {
    System.out.println(dist[n] + 1);
}

sc.close();
}
}

```

Why +1? dist stores number of edges, but the question asks for number of servers including start and destination.

20. Minimum Distance Between Two Nodes

Task: Find minimum number of edges between nodes x and y in an undirected unweighted graph.

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        int x = sc.nextInt();
        int y = sc.nextInt();

        int[] dist = new int[n + 1];
        Arrays.fill(dist, -1);

        Queue<Integer> q = new LinkedList<>();
        q.add(x);
        dist[x] = 0;

        while (!q.isEmpty()) {
            int node = q.poll();

            for (int nei : graph.get(node)) {
                if (dist[nei] == -1) {
                    dist[nei] = dist[node] + 1;
                    q.add(nei);
                }
            }
        }

        System.out.println(dist[y]);

        sc.close();
    }
}

```

Time: $O(n + m)$

Space: $O(n + m)$

21. Count Connected Components

Task: Count the number of connected components in an undirected graph.

```

import java.util.*;

class Main {
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph) {
        visited[node] = true;

        for (int nei : graph.get(node)) {
            if (!visited[nei]) {
                dfs(nei, visited, graph);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();

```



```

        int v = sc.nextInt();
        graph.get(u).add(v);
        graph.get(v).add(u);
    }

    boolean[] visited = new boolean[n + 1];
    int components = 0;

    for (int i = 1; i <= n; i++) {
        if (!visited[i]) {
            components++;
            dfs(i, visited, graph);
        }
    }

    System.out.println(components);

    sc.close();
}
}

```

Time: $O(n + m)$

Space: $O(n + m)$

22. Roads Construction In Chef's Country

Task: Find the minimum number of roads needed to connect all cities.

- A. connected components
- B. connected components - 1
- C. number of nodes - 1
- D. number of edges + 1

Answer: B

Explanation: If there are k connected components, we need $k - 1$ new roads to connect all components.

Code idea: Count connected components using DFS/BFS, then print components - 1.

23. Galactic Network

Task: Find the number of disconnected groups in an undirected graph.

- A. Number of edges
- B. Number of connected components
- C. Number of cycles
- D. Number of isolated edges

Answer: B

Explanation: Each disconnected group is one connected component.

Code idea: Same as connected components count using DFS/BFS.

Part 4: Extra Coding Problems

24. LeetCode 3160 - Find The Number Of Distinct Colors Among The Balls

Pattern: HashMap + frequency count

We need to track:

- each ball's current color
- how many balls currently have each color
- number of distinct colors after every query

```

import java.util.*;

class Solution {
    public int[] queryResults(int limit, int[][] queries) {
        HashMap<Integer, Integer> ballColor = new HashMap<>();
        HashMap<Integer, Integer> colorCount = new HashMap<>();

        int n = queries.length;
        int[] result = new int[n];

        for (int i = 0; i < n; i++) {
            int ball = queries[i][0];
            int newColor = queries[i][1];

            if (ballColor.containsKey(ball)) {

```

```

        int oldColor = ballColor.get(ball);

        colorCount.put(oldColor, colorCount.get(oldColor) - 1);

        if (colorCount.get(oldColor) == 0) {
            colorCount.remove(oldColor);
        }

        ballColor.put(ball, newColor);
        colorCount.put(newColor, colorCount.getDefault(newColor, 0) + 1);

        result[i] = colorCount.size();
    }

    return result;
}
}

```

Logic:

```

ballColor: ball -> current color
colorCount: color -> number of balls with that color

```

Example:

```

queries = [[1,4],[2,5],[1,3],[3,4]]
After [1,4]: colors {4} -> 1
After [2,5]: colors {4,5} -> 2
After [1,3]: ball 1 changes from 4 to 3, colors {3,5} -> 2
After [3,4]: colors {3,5,4} -> 3
Output: [1, 2, 2, 3]

```

Time: O(n)

Space: O(n)

25. Sum Numbers In Alphanumeric String

This is not sum of digits. It is the sum of full numbers inside the string.

Examples:

```

abc123xyz -> 123
10a20b30 -> 10 + 20 + 30 = 60
00abc12ghj -> 0 + 12 = 12
5 5 5 -> 5 + 5 + 5 = 15
abcDEFghj -> 0

```

```

class Main {
    public static void main(String[] args) {
        String s = "10a20b30";

        long sum = 0;
        long num = 0;
        boolean buildingNumber = false;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (Character.isDigit(ch)) {
                num = num * 10 + (ch - '0');
                buildingNumber = true;
            } else {
                if (buildingNumber) {
                    sum += num;
                    num = 0;
                    buildingNumber = false;
                }
            }
        }

        if (buildingNumber) {
            sum += num;
        }

        System.out.println(sum);
    }
}

```

Pattern:

```

If digit -> build number
If non-digit -> add completed number to sum
At end -> add last number if present

```

Time: O(n)

Space: O(1)

Final Graph Revision Notes

- Adjacency matrix memory: $O(V^2)$
- Adjacency list memory: $O(V + E)$

- DFS uses recursion or stack
- BFS uses queue
- BFS gives shortest path in unweighted graphs
- Connected components are counted using DFS/BFS from every unvisited node
- Minimum roads to connect all components = components - 1
- Tree = connected graph with no cycles
- Tree with n nodes has n - 1 edges
- Complete graph with n nodes has $n(n - 1) / 2$ edges

Veeva Optimized Coding Set - Java

This sheet contains optimized Java solutions for the coding problems shared for Veeva preparation.

1. Make String a Subsequence Using Cyclic Increments

Pattern: Two pointers

Idea: Each character in str1 can either stay same or increment once cyclically. So str1[i] can match str2[j] if:

- str1[i] == str2[j]
- or next cyclic character of str1[i] equals str2[j]

```
class Solution {
    public boolean canMakeSubsequence(String str1, String str2) {
        int j = 0;

        for (int i = 0; i < str1.length() && j < str2.length(); i++) {
            char current = str1.charAt(i);
            char next = current == 'z' ? 'a' : (char) (current + 1);

            if (current == str2.charAt(j) || next == str2.charAt(j)) {
                j++;
            }
        }

        return j == str2.length();
    }
}
```

Time: O(n)

Space: O(1)

2. Invalid Transactions

Pattern: String parsing + comparison

Standard LeetCode rule: A transaction is invalid if:

- amount > 1000
- or same name has another transaction within 60 minutes in a different city

```
import java.util.*;

class Solution {
    static class Transaction {
        String original;
        String name;
        int time;
        int amount;
        String city;

        Transaction(String transaction) {
            original = transaction;
            String[] parts = transaction.split(",");
            name = parts[0];
            time = Integer.parseInt(parts[1]);
            amount = Integer.parseInt(parts[2]);
            city = parts[3];
        }
    }

    public List<String> invalidTransactions(String[] transactions) {
        int n = transactions.length;
        Transaction[] arr = new Transaction[n];

        for (int i = 0; i < n; i++) {
            arr[i] = new Transaction(transactions[i]);
        }

        boolean[] invalid = new boolean[n];
```

```

        for (int i = 0; i < n; i++) {
            if (arr[i].amount > 10000) {
                invalid[i] = true;
            }

            for (int j = i + 1; j < n; j++) {
                if (arr[i].name.equals(arr[j].name)
                    && !arr[i].city.equals(arr[j].city)
                    && Math.abs(arr[i].time - arr[j].time) <= 60) {
                    invalid[i] = true;
                    invalid[j] = true;
                }
            }
        }

        List<String> result = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (invalid[i]) {
                result.add(arr[i].original);
            }
        }

        return result;
    }
}

```

Time: $O(n^2)$

Space: $O(n)$

Note: This is accepted because constraints are small in the original problem.

3. Subarrays With K Different Integers

Pattern: Sliding window + HashMap

Formula:

```

exactly(k) = atMost(k) - atMost(k - 1)

import java.util.*;

class Solution {
    public int subarraysWithKDistinct(int[] nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);
    }

    private int atMost(int[] nums, int k) {
        if (k == 0) return 0;

        HashMap<Integer, Integer> map = new HashMap<>();
        int left = 0;
        int count = 0;

        for (int right = 0; right < nums.length; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

            while (map.size() > k) {
                map.put(nums[left], map.get(nums[left]) - 1);
                if (map.get(nums[left]) == 0) {
                    map.remove(nums[left]);
                }
                left++;
            }

            count += right - left + 1;
        }

        return count;
    }
}

```

Time: $O(n)$

Space: $O(k)$

4. Shortest Path in Binary Matrix

Pattern: BFS in 8 directions

```

import java.util.*;

class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;

        if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) {
            return -1;
        }

        int[][] dirs = {
            {-1, -1}, {-1, 0}, {-1, 1},
            {0, -1}, {0, 1},
            {1, -1}, {1, 0}, {1, 1}
        };

        Queue<int[]> q = new LinkedList<>();
        q.add(new int[]{0, 0, 1});
        grid[0][0] = 1;
    }
}

```

```

        while (!q.isEmpty()) {
            int[] curr = q.poll();
            int r = curr[0];
            int c = curr[1];
            int dist = curr[2];

            if (r == n - 1 && c == n - 1) {
                return dist;
            }

            for (int[] d : dirs) {
                int nr = r + d[0];
                int nc = c + d[1];

                if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] == 0) {
                    grid[nr][nc] = 1;
                    q.add(new int[]{nr, nc, dist + 1});
                }
            }
        }

        return -1;
    }
}

```

Time: $O(n^2)$

Space: $O(n^2)$

5. Find All Duplicates in an Array

Pattern: Index marking / sign marking

Condition: numbers are in range $[1, n]$, each appears at most twice.

```

import java.util.*;

class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < nums.length; i++) {
            int index = Math.abs(nums[i]) - 1;

            if (nums[index] < 0) {
                result.add(index + 1);
            } else {
                nums[index] = -nums[index];
            }
        }

        return result;
    }
}

```

Time: $O(n)$

Extra Space: $O(1)$, excluding output

6. Duplicate Objects in an Array of Objects

In Java, represent each object as a User. Duplicate is based on name + email.

Pattern: HashMap with composite key

```

import java.util.*;

class User {
    int id;
    String name;
    String email;

    User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public String toString() {
        return id + " " + name + " " + email;
    }
}

class Main {
    public static void main(String[] args) {
        User[] users = {
            new User(1, "Amit Kumar", "amit@example.com"),
            new User(2, "Sumit Kumar", "sumit@example.com"),
            new User(3, "Amit Kumar", "amit@example.com"),
            new User(4, "Raj Kumar", "raj@example.com"),
            new User(5, "Amit Kumar", "amit@example.com")
        };

        HashMap<String, List<User>> map = new HashMap<>();

        for (User user : users) {
            String key = user.name + "#" + user.email;
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(user);
        }
    }
}

```

```

    }

    List<User> duplicates = new ArrayList<>();

    for (List<User> group : map.values()) {
        if (group.size() > 1) {
            duplicates.addAll(group);
        }
    }

    for (User user : duplicates) {
        System.out.println(user);
    }
}
}

```

Time: $O(n)$

Space: $O(n)$

7. Duplicates in Limited Range and Limited Repetition Array

Same optimized idea as Problem 5.

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {2, 3, 1, 2, 3};
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < arr.length; i++) {
            int index = Math.abs(arr[i]) - 1;

            if (arr[index] < 0) {
                result.add(index + 1);
            } else {
                arr[index] = -arr[index];
            }
        }

        System.out.println(result);
    }
}

```

Time: $O(n)$

Extra Space: $O(1)$, excluding output

8. Longest Increasing Subsequence

Optimized pattern: Binary search + tails array

```

class Solution {
    public int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;

        for (int x : nums) {
            int left = 0;
            int right = size;

            while (left < right) {
                int mid = left + (right - left) / 2;

                if (tails[mid] < x) {
                    left = mid + 1;
                } else {
                    right = mid;
                }
            }

            tails[left] = x;

            if (left == size) {
                size++;
            }
        }

        return size;
    }
}

```

Time: $O(n \log n)$

Space: $O(n)$

9. LIS in Circular Manner

Constraints: $N \leq 300$, so $O(n^3)$ is acceptable.

Idea:

- Try every starting position.
- Build the circular array of length n from that start.
- Compute normal LIS using $O(n^2)$.
- Take maximum.

```
public class Solution {
    public static int lisInCircularManner(int n, int[] arr) {
        int answer = 1;

        for (int start = 0; start < n; start++) {
            int[] circular = new int[n];

            for (int i = 0; i < n; i++) {
                circular[i] = arr[(start + i) % n];
            }

            int[] dp = new int[n];
            for (int i = 0; i < n; i++) {
                dp[i] = 1;
            }

            int best = 1;

            for (int i = 0; i < n; i++) {
                for (int j = 0; j < i; j++) {
                    if (circular[j] < circular[i]) {
                        dp[i] = Math.max(dp[i], dp[j] + 1);
                    }
                }
                best = Math.max(best, dp[i]);
            }

            answer = Math.max(answer, best);
        }

        return answer;
    }
}
```

Time: $O(n^3)$

Space: $O(n)$

Since n is only 300, this is safe for the given constraints.

10. Minimum Loss From Buying and Selling Once

Problem: Given prices on each day, buy once and sell once later. Find minimum possible loss.

Important: Buy day must come before sell day. Loss means buy price > sell price and loss = buy - sell.

Examples:

```
[6, 8, 10, 4, 9] -> 1
Buy at 10, sell at 9, loss = 1

[8, 9, 3, 5] -> 3
Buy at 8, sell at 5, loss = 3
```

Optimized approach: TreeSet

While scanning prices from left to right, TreeSet stores previous buy prices. For current sell price, find the smallest previous price greater than current price.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] prices = {6, 8, 10, 4, 9};

        TreeSet<Integer> previousPrices = new TreeSet<>();
        int minLoss = Integer.MAX_VALUE;

        for (int price : prices) {
            Integer higher = previousPrices.higher(price);

            if (higher != null) {
                minLoss = Math.min(minLoss, higher - price);
            }

            previousPrices.add(price);
        }

        System.out.println(minLoss);
    }
}
```

Time: $O(n \log n)$

Space: $O(n)$

Quick Pattern Summary

1. Cyclic subsequence - Two pointers
2. Invalid transactions - Parsing + comparison
3. Subarrays with K distinct - Sliding window
4. Binary matrix shortest path - BFS
5. Duplicates in range 1 to n - Sign marking
6. Duplicate objects - HashMap composite key
7. Limited range duplicates - Sign marking
8. LIS - Binary search
9. Circular LIS - Try all rotations + LIS DP
10. Minimum loss - TreeSet

Latest SQL and Coding Additions

SQL: Sum Salaries Of Employees With No Completed Projects

Requirement: Find the sum of salaries of employees who were assigned at least one project and none of their projects have an End_dt.

```
WITH slack_employees AS (
    SELECT
        e.id,
        e.salary
    FROM employees e
    JOIN projects p
    ON e.id = p.employee_id
    GROUP BY e.id, e.salary
    HAVING COUNT(p.project_id) >= 1
    AND SUM(CASE WHEN p.End_dt IS NOT NULL THEN 1 ELSE 0 END) = 0
)
SELECT SUM(salary) AS total_slack_salary
FROM slack_employees;
```

Pattern: group by employee, filter employees with zero completed projects, then sum salaries.

Coding: Frequency Of Each String

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        String[] arr = {"apple", "banana", "apple", "cherry", "banana", "apple"};

        HashMap<String, Integer> freq = new HashMap<>();

        for (String s : arr) {
            freq.put(s, freq.getOrDefault(s, 0) + 1);
        }

        System.out.println(freq);
    }
}
```

Time: $O(n)$ average

Space: $O(k)$, where k is number of unique strings.

Coding: Valid Parentheses

```
import java.util.*;

class Main {
    static boolean isBalanced(String s) {
        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else {
                if (stack.isEmpty()) return false;

                char top = stack.pop();

                if (ch == ')' && top != '(') return false;
                if (ch == '}' && top != '{') return false;
                if (ch == ']' && top != '[') return false;
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {
```



```

        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();

        while (t-- > 0) {
            String str = sc.next();
            System.out.println(isBalanced(str) ? "Balanced" : "Not Balanced");
        }

        sc.close();
    }
}

```

Time: O(n) per string

Space: O(n)

Coding: Palindrome Linked List

```

class Solution {
    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;

        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode secondHalf = reverse(slow);
        ListNode firstHalf = head;

        while (secondHalf != null) {
            if (firstHalf.val != secondHalf.val) return false;
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
        }

        return true;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            ListNode nextNode = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextNode;
        }

        return prev;
    }
}

```

Time: O(n)

Space: O(1)

Coding: Spiral Level Order Traversal Of Binary Tree

```

import java.util.*;

class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);
        boolean leftToRight = true;

        while (!queue.isEmpty()) {
            int size = queue.size();
            LinkedList<Integer> level = new LinkedList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();

                if (leftToRight) {
                    level.addLast(node.val);
                } else {
                    level.addFirst(node.val);
                }

                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }

            result.add(level);
            leftToRight = !leftToRight;
        }

        return result;
    }
}

```

Time: O(n)

Space: $O(n)$